

Understand your commerce architecture

A foundation-first study guide for the database, backend, frontend, and engineering decisions in your assessment project.

What this guide is for

Use this to understand the implementation well enough to explain it yourself. It teaches vocabulary, connects concepts to actual files, and asks you to trace behavior. It is AI-generated learning material, not the manually authored documentation requested by the assessment.

The assessment explicitly requires both the architectural documentation and the AI-methodology document to be written in your own words without AI assistance. Do not copy or paraphrase this guide into those deliverables. Close it, inspect the code, and write from your own understanding. If the rules also restrict AI-assisted study, clarify that with the assessor.

How to study

Read pages 2-3 first. Work through the database on pages 4-7, backend on pages 8-10, and frontend on pages 11-13. Pages 14-16 cover testing, reasoning about choices, and independent practice. Open the referenced files as you read; paths are relative to the repository root.

What is verified, and what is interpretation

Implementation descriptions come from the working-tree source inspected for this guide on 27 September 2026. Reasons such as simplicity or maintainability are engineering interpretations of those choices, not proof of what you personally intended. Only claim a motivation as yours if it reflects your actual thinking.

Current stack: Next.js 16 and React 19 for the storefront; Express 5 and TypeScript for the API; Sequelize with SQLite for persistence. Images are stored as database BLOBs. Cloudflare R2 is not part of the current image-delivery path.

Learning outcome

You should be able to follow a purchase from a button click to committed database rows, explain which layer owns each decision, and identify the limitations of the implementation without describing it as more complete than it is.

The mental model: three domains

Frontend: interaction and presentation

The frontend decides what the customer sees and how they interact with it: forms, product cards, navigation, loading states, and error messages. React turns state into UI. Next.js also runs server-side code, so frontend does not mean that every file executes in the browser.

Backend: trusted application behavior

The Express API accepts HTTP requests, validates input, authenticates users, checks business rules, and coordinates database work. It decides whether an action is allowed and calculates the authoritative order price. A browser can send altered requests, so the backend cannot trust the UI to enforce rules.

Database: durable records and relationships

SQLite stores accounts, products, variants, cart lines, wishlist entries, orders, order items, and image bytes. Durable means the data survives a browser refresh or process restart when the database file is retained. React state alone does not provide this persistence.

```
Browser -> Next.js UI / server rendering  
Browser or Next.js server -> Express HTTP API  
Express -> Sequelize -> SQLite file
```

This is one Git repository with two application runtimes. The root scripts delegate to apps/api and apps/web. A monorepo is a source organization choice; it does not imply that everything runs as one process or must deploy together.

The boundary matters

The web app consumes an API contract, not Sequelize models. A contract describes the request and response shape. A DTO (data transfer object) is the selected representation crossing that boundary: for example a product response with a dollar price, image path, and variants.

Explain it back

Where should a product card animation live? Where should stock be decremented? What survives if the customer closes the browser? Answer each by naming the responsible domain.

Open in your repository: package.json; apps/api/src/app.ts; apps/web/src/services/server-api.js

HTTP and one complete request

HTTP is the conversation between a client and a server. A request has a method, URL, headers, and sometimes a body. A response has a status code, headers, and a body. JSON represents structured data; an image endpoint returns binary bytes instead.

Methods and status codes in context

GET reads data. **POST** creates or performs an action. **PATCH** changes part of a resource. **DELETE** removes it. Typical results include 200 success, 201 created, 400 invalid input, 401 missing/invalid authentication, 403 forbidden, 404 missing resource, 409 conflict, and 500 unexpected failure.

Trace: adding a cart item

1. A product component calls the cart provider with a product ID and variant ID.
2. The provider calls a cart service, which calls the shared browser API client.
3. The browser sends JSON and includes the session cookie.
4. Express middleware handles the request boundary; route middleware authenticates and validates it.
5. The controller passes the authenticated user identity and input to the service.
6. The service checks the product, variant, quantity, and applicable rules, then uses a repository.
7. Sequelize reads or writes SQLite records.
8. The API returns cart data; the provider updates React state; the UI renders the new quantity and totals.

Why not send everything from the browser?

The browser can identify the requested item, but a submitted price or user ID is not a trustworthy authority. Ownership must come from the verified session. The backend reads the actual price and stock from its own records.

Asynchronous code

A Promise represents a result that will arrive later. `await` pauses that function until the Promise settles; it does not make network or database operations instant. The UI needs pending and error states because a request can be slow or fail.

Explain it back

Use the files below to find the exact add-cart URL and body. Then identify where the current user ID comes from. Could changing a browser variable legitimately make you another customer?

Open in your repository: `apps/web/src/services/cart-service.js`; `apps/web/src/services/api-client.js`; `apps/api/src/routes/cart.routes.ts`; `apps/api/src/services/cart.service.ts`

Database basics and your schema

A relational database organizes records in tables. A row is one record; a column is one field. A primary key identifies a row. A foreign key links a record to another table and can prevent references to records that do not exist. A unique constraint prevents duplicate values in a specified column or combination.

Table	Responsibility
users	Identity, email, password hash
products	Catalog details, price in cents, stock, image key
product_variants	Options belonging to a product
cart_items	A user/product/variant combination and quantity
wishlist_items	A user/product pair
orders	Customer and delivery details, totals, order status
order_items	Purchased names, variants, quantities and prices
assets	Image key, MIME type, bytes and ETag
schema_migrations	Which schema migrations have already run

Relationships

One product has many variants. One user can have many cart entries and orders. One order has many order items. A cart line includes a variant so two options of the same product can be separate lines. Wishlist membership is a user/product relationship with no quantity.

The product image value is an application-level asset key. The current schema does not declare it as a foreign key to assets. Do not describe a database constraint that the migration does not actually define.

Explain it back

Why is email unique? Why does a cart line need more than a product ID? What would break if a variant could refer to a nonexistent product?

Open in your repository: `apps/api/src/database/migrations/001-initial-schema.ts`; `migrations/007-assets.ts`; `database/models/index.ts` (under `apps/api/src`)

Models, migrations, and seeds

Three different jobs

A **model** describes how application code works with a table: fields, types, and associations. A **migration** changes the database structure in an ordered, tracked step. A **seed** inserts or updates starter data. A model edit alone does not add a column to an existing database.

Your startup sequence

```
configureDatabase()  
-> runMigrations()  
-> initializeModels()  
-> seedDatabase()
```

The migration runner reads `schema_migrations`, skips recorded migrations, and applies each pending migration within a transaction. It records the migration ID alongside the schema change. The server does not rely on automatic `sync({ alter: true })` to guess schema updates.

Idempotent does not mean unchanged

Running a seed repeatedly should not create duplicate starter records. Your seed uses stable product and variant IDs, `findOrCreate`, updates, and asset upserts. Existing product names, prices, descriptions, and image keys can be refreshed. Existing stock quantities are deliberately not reset by the update branch, so sold stock stays sold.

All 19 source images are read from `apps/api/assets` and upserted into the database during seeding. Startup currently runs this seed too. Consequently, those source files remain a startup dependency, even though individual image requests read the database. At larger scale, repeatedly importing images at startup would be work worth separating into an explicit import step.

SQLite and Sequelize

SQLite is the database engine and file. Sequelize is the ORM: it maps JavaScript/TypeScript operations to database queries. The ORM reduces repeated SQL handling, but you still need to understand constraints, transactions, query cost, and which associations have been loaded.

Explain it back

If you add a new delivery field, which model, migration, validation contract, and serializer might need changes? Why would deleting the database lose customer accounts even though the seed can recreate products?

Open in your repository: `apps/api/src/database/bootstrap.ts`; `seed.ts`; `migrations/index.ts`; `apps/api/src/config/database.ts`

Money, transactions, and stock

Store money as integer cents

Binary floating-point numbers cannot exactly represent every decimal fraction. Your database stores 240 dollars as 24000 cents. Calculation uses integer cents; serializers convert to dollars for the API. This makes the stored monetary unit explicit and avoids many rounding surprises. A full financial system still needs explicit rounding and currency rules.

A transaction is an all-or-nothing unit

Suppose an order contains two products. Decreasing stock for the first product and then failing on the second must not leave a half-finished purchase. A transaction groups the changes: commit keeps them; rollback undoes them. Your checkout places stock reservation, order creation, order-item creation, and cart clearing in one transaction.

A stock check must be part of the update

```
UPDATE products
SET stock_quantity = stock_quantity - requested_quantity
WHERE id = product_id
AND stock_quantity >= requested_quantity
```

This is conceptual SQL for the guarded update in `order.repository.ts`. The code checks that exactly one row was affected. If none was updated, checkout reports insufficient stock. A separate read followed by an unconditional decrement would allow competing requests to act on stale stock.

A concrete boundary example

The current shipping policy charges \$6 for a positive shipped subtotal at or below \$50. Shipping is free above \$50. Pickup has no shipping charge. Therefore \$50.00 and \$50.01 have different shipping behavior. Read the comparison operator before explaining a boundary in words.

SQLite supports transactions, but it is not an unlimited concurrent write system. WAL improves reader/writer coexistence; writes still have contention. A database transaction also cannot automatically undo an external payment or email.

Explain it back

What happens if `createItems` throws after stock was reserved? Why should a forged subtotal in a browser request have no effect on the final order total?

Open in your repository: `apps/api/src/services/order.service.ts`; `repositories/order.repository.ts`; `policies/commerce.policy.ts`

Images and historical order data

Where the image actually lives

A BLOB is a binary large object: the actual file bytes stored in a database column. The assets table stores the image data, its content type (such as image/jpeg), its key, and an ETag derived from a SHA-256 hash. The product row stores only the key, not another copy of the image bytes.

```
Seed source: apps/api/assets/images/products/...jpg
-> assets.data (SQLite BLOB)
GET /api/assets/images/products/...jpg
-> Asset.findByPk(key) -> binary HTTP response
```

The public product DTO exposes /assets/... relative to the API base. The frontend asset helper prefixes NEXT_PUBLIC_API_URL. Next Image can then optimize the fetched image through its own image endpoint. A URL is an address for obtaining bytes; its existence does not mean the original bytes are stored at that URL as a disk file.

Caching saves repeated transfer

Content-Type tells the client how to interpret bytes. ETag identifies a version. A client can send If-None-Match; when the cached version is still valid, the server can return 304 without the image body. The current cache header allows one day of freshness and a stale-while-revalidate interval. Changes to a stable image key may remain cached during the freshness period.

The current implementation loads a BLOB into a Buffer and sends it. It does not incrementally stream from SQLite. Database storage simplifies the assessment setup and backup scope, but increases database size, backup cost, and API traffic. Object storage is a possible future alternative, not a current dependency.

Why order items copy names and prices

Catalog data describes what is on sale now. Order items describe what was bought then. Copying the purchased name, variant name, unit price, and line total preserves history when a product is renamed or repriced. This deliberate duplication serves a different purpose from redundant source files.

Explain it back

If an image is missing, which endpoint and table should you inspect? Why must an old order total stay unchanged when the catalog price changes?

Open in your repository: apps/api/src/controllers/asset.controller.ts; database/models/asset.model.ts; apps/web/src/lib/assets.js; apps/web/next.config.mjs

Backend layers: who owns what?

```
Route -> Controller -> Service -> Repository -> Model
|
+--> Policy (pure business calculation)
```

Route and middleware

A route associates a method and URL with handlers. Middleware runs around handlers for concerns such as logging, cookies, authentication, rate limits, and input validation. Order matters: a protected controller must not run as if an unverified cookie were an authenticated user.

Controller

The controller translates HTTP into application calls and responses. It reads validated input and the authenticated user, calls a service, and chooses the response shape/status. Your product-list controller directly parses query parameters. Most mutation routes use the shared validation middleware.

Service

A service expresses a use case: register, add a cart line, or place an order. It coordinates repositories and owns multi-step business behavior. Checkout is a strong example because several database writes must succeed together. Putting that workflow in a controller would mix HTTP mechanics with business rules.

Repository and model

A repository names database operations and keeps query details together. A model maps those operations to stored fields and relationships. This split adds files, so its value is clearer ownership and testability, not fewer lines. The small asset read goes straight from its controller to a repository because it has no substantial business workflow to coordinate.

Policy and serializer

A policy function calculates or decides without handling HTTP. Pure functions are easy to test because their output depends on their input. A serializer selects public fields, translates values, and avoids leaking internal columns such as password hashes. Internal model shape and external API shape need not match.

Explain it back

Where would a new free-shipping rule go? Where would you change a SQL include? Where would you decide whether an error should become HTTP 409?

Open in your repository: apps/api/src/routes; controllers; services; repositories; policies; serializers

Authentication and authorization

Different questions

Authentication asks who is making the request. **Authorization** asks whether that identity may perform the action on that resource. A signed-in user is not entitled to read another customer's cart. Ownership must be enforced by queries using the authenticated user ID.

Registration and login

Registration validates input, checks email availability, hashes the password with bcrypt at cost 12, and creates a user. A unique database constraint remains necessary because two registrations could pass an initial availability check at the same time. Login compares the submitted password with the stored hash; it does not decrypt a stored password.

Token and cookie

The backend signs a JWT with HS256. The token subject contains the user ID and expires after seven days. A signature lets the backend detect tampering; it does not encrypt the payload. The controller stores the token in an HttpOnly cookie. HttpOnly prevents ordinary page JavaScript from reading the cookie; Secure is enabled in production, and SameSite is Lax.

The browser API client uses credentials: include. The API verifies the signature and expiry and looks up the user before attaching request.user. The Next.js server adapter forwards cookies when making authenticated API requests. A cookie merely being present is not proof that its contents are valid.

What logout does, and does not do

Logout clears the browser cookie. The current design has no server-side token revocation list. A previously copied valid token can remain usable until expiration. Also, choosing a session cookie instead of a persistent cookie does not change the signed token's seven-day expiry.

Frontend navigation is not the authority

src/proxy.js checks cookie presence to redirect routes. The API independently verifies every protected request. A stale cookie can mislead the navigation check, which is why authoritative authentication must stay in the API.

Explain it back

Why is a password hash different from encryption? What happens if someone invents a cookie with the correct name? Why is authorization still needed after a valid login?

Open in your repository: `apps/api/src/services/auth.service.ts`; `security/session.ts`; `middleware/authenticate.ts`; `apps/web/src/proxy.js`

Validation, errors, and configuration

Types do not validate a network request

TypeScript checks code during development; its types disappear at runtime. JSON sent by a client can contain anything. Zod contracts validate incoming shapes and rules at runtime. Browser form validation improves feedback, but the server must validate the same untrusted request independently.

CORS and origin checks

CORS tells a browser which origins may read responses. It is not authentication and does not stop every non-browser client. Your API uses an origin allowlist and credentials. A separate origin guard rejects mutating requests with a disallowed Origin; the current code allows requests without an Origin header. Describe this accurately rather than claiming every request requires a trusted Origin.

Consistent failures

Known `AppError` instances become explicit status codes and error objects. Unexpected errors are logged and return a generic 500 message. This keeps internal database details out of the public response. The frontend API client turns failed responses into an `ApiError` so components can display useful feedback.

Configuration has a home

The API uses its own environment file for its database location, port, allowed origins, and authentication secret. The web app has separate browser and server API URLs. `NEXT_PUBLIC` variables are exposed to browser code and embedded during the build; never place a secret in them. An `.env.example` is a template, not a real credential store.

A root Prettier configuration shares formatting rules. Separate ESLint configurations express different needs: TypeScript-aware API rules and Next.js/React frontend rules. One filename appearing in two apps is not automatically wasteful duplication if its behavior differs.

Input size and observability

The app also uses Helmet, request-size limits, rate limits, and structured request logging. These are layers with specific purposes; they do not prove the whole system secure. A request ID helps connect a failed response to its server log entry.

Explain it back

Why is `AUTH_SECRET` server-only? Why can CORS not prove which user owns a cart? Which error details help the customer, and which should stay in logs?

Open in your repository: `apps/api/src/config/env.ts`; `middleware/validate.ts`; `middleware/origin-guard.ts`; `middleware/error-handler.ts`; `apps/web/src/services/api-client.js`

React and Next.js foundations

Components, props, state

A component describes a part of the interface. Props are values supplied by its parent, such as a product. State is data the component retains and changes, such as whether a quick-view panel is open. Updating state causes React to calculate the next UI. Rendering should not itself perform uncontrolled writes or purchases.

Server and client components

App Router components are server components by default. A client boundary marked with "use client" enables hooks, event handlers, and browser behavior. Client components can still be prerendered; the directive does not mean the initial page can never involve the server. It determines a client module boundary and its dependencies.

Your root layout resolves the current session through the server adapter. Product pages fetch catalog data on the server. Interactive components and providers use client state. The server-only import in server-api.js helps prevent accidentally using that adapter in a client module.

Rendering and hydration

Server rendering prepares content before the browser handles interaction. Hydration attaches React behavior to the rendered client UI. Reading browser-only APIs during inappropriate rendering stages can cause mismatches or failures. Effects are useful for synchronizing client state with external operations after rendering.

Routing conventions

app/products/page.jsx defines /products. app/products/[slug]/page.jsx handles a specific product path. layout.jsx wraps descendants; loading.jsx provides loading UI; error.jsx and global-error.jsx handle error boundaries. robots.js, sitemap.js, and favicon.ico use framework conventions rather than arbitrary organization.

The cart, checkout, and wishlist pages are client components. Their small layout files export metadata from server components. Returning children alone is valid when the layout exists to carry metadata.

Explain it back

Why does a button with an onClick handler need client behavior? Why should a database password never be imported into a product card? What does [slug] represent?

Open in your repository: apps/web/src/app/layout.jsx; app/products/page.jsx; app/products/[slug]/page.jsx; app/cart/layout.jsx; services/server-api.js

State, API adapters, and user feedback

Shared state with Context

Context allows descendants to consume a shared value without passing it through every intermediate component. Your auth, cart, wishlist, and currency providers group related state and actions. The cart provider fetches persisted lines when mounted and replaces its local lines with the API response after a successful mutation.

This is not a full optimistic-update system. In a typical optimistic update the UI changes before the server confirms it and rolls back on failure. Here, key cart actions await the API and then update state. Loading flags and disabled controls tell the customer that work is still pending.

Persistence versus display state

Cart and wishlist records persist in SQLite. An open dropdown or quick-view dialog is temporary component state. Currency choice in the current provider is React state; rates are hard-coded display conversions rather than a live exchange-rate service. Order calculations remain based on the backend's stored money values.

Why two API adapters?

api-client.js serves browser requests and includes credentials. server-api.js runs in Next.js server code, can use a private BACKEND_API_URL, and forwards the incoming cookie when needed. Product and auth services name feature actions over those transport details. Separating them makes the execution context visible.

Handle the full request lifecycle

A complete screen considers loading, success, empty data, and failure. Zero cart items is not the same as a failed cart request. Your providers expose loading/error state, and reusable UI components such as PageLoader, EmptyState, and ErrorView keep that feedback consistent.

useMemo caches a calculated value when dependencies do not change; useCallback stabilizes a function reference. Neither makes the server response trustworthy, and neither should be described as a universal performance improvement without measuring the relevant behavior.

Explain it back

Which data would survive signing out and back in? What happens if addItem rejects? Why might the server reach the API at a different URL from the browser?

Open in your repository: apps/web/src/components/providers/cart-provider.jsx; currency-provider.jsx; apps/web/src/services/api-client.js; server-api.js

Folder structure and frontend design

The current web structure is sensible for an assessment. There is no single optimal folder arrangement: a useful structure makes ownership clear and common changes easy to locate.

```
apps/web/  
src/app/ routes, layouts, metadata  
src/components/ feature UI, providers, shared UI  
src/services/ HTTP clients and feature requests  
src/lib/ calculation and formatting helpers  
src/styles/ shared CSS module  
src/proxy.js navigation checks  
e2e/ real browser journeys  
test/ unit-test setup and test shims
```

Colocation and CSS Modules

Many components keep their CSS module and test beside the implementation. This makes related changes easier to find. CSS Modules scope class names, reducing accidental global collisions. `globals.css` handles application-wide styling. Feature folders such as `cart` and `product group` components by responsibility.

Small improvements supported by the code

`product-detail.module.css` is in `components/product` although its consumer is the product detail page; placing it beside that page would make ownership clearer. `product-catalog.module.css` also supplies wishlist styling, which couples the wishlist to catalog implementation details. A small shared grid style could express that reuse more explicitly.

`ProductCatalog` is approximately 600 lines and combines several controls, responsive UI, and catalog behavior. Extracting cohesive filters, toolbar, or pagination pieces would help readability. File length is a clue to inspect responsibilities, not a rule demanding an arbitrary split.

`AppShell` branches on pathname to choose providers and chrome. Route groups such as `(auth)` and `(shop)` could express shared layouts more directly as the app grows. This is a possible refactor, not a required fix. Adding `features/`, `hooks/`, or more layers solely for appearance would not automatically improve the project.

Explain it back

Find one component used by multiple routes and one used by only one route. Which deserves a shared location? Can you explain the present structure without claiming it is the only valid Next.js structure?

Open in your repository: `apps/web/src/components/layout/app-shell.jsx`; `components/product/product-catalog.jsx`; `app/products/[slug]/page.jsx`; `vitest.config.mjs`

Testing and implementation flow

Different tests answer different questions

Unit tests isolate calculations or component behavior: for example shipping boundaries or a button state. **Integration tests** exercise collaborating parts: the API tests use Supertest with the app and database setup. **End-to-end tests** use Playwright with both apps to exercise customer journeys in a browser.

Formatting checks style. ESLint identifies rule violations and suspicious patterns. Type checking checks compile-time consistency. A production build checks whether the app can compile for deployment. These checks complement tests; none alone proves correctness or security.

What the repository currently demonstrates

Earlier verification in this session reported 22 API tests, 28 frontend tests, and four Playwright journeys passing after the image-storage change. That is a point-in-time result, not a fresh rerun for this PDF. Test counts can change. Read the assertions before claiming that a particular behavior is covered.

A useful order for understanding development

Start with the requirement and invariants. Model records and relationships. Define request/response contracts. Implement database changes and business rules. Connect HTTP routes. Build UI against the contract. Exercise success and failure paths. Review the result and document what actually happened.

This sequence is a learning model, not a claim about the exact order in which you personally developed the project. Use Git history, your notes, and the conversation record to reconstruct the real chronology.

Useful verification commands

```
npm run check --prefix apps/api
npm run check --prefix apps/web
npm run test:e2e --prefix apps/web
```

Run these from the repository root. The E2E setup builds and starts both apps with an isolated temporary SQLite database. A green journey checks its specific assertions; it is not evidence that every possible user action was tested.

Explain it back

Which test would you add for a \$50.00 shipping boundary? Which test would reveal a broken browser/API connection? Why does passing a build not prove checkout reserves stock correctly?

Open in your repository: `apps/api/test/api.integration.test.ts`; `src/policies/commerce.policy.test.ts`; `apps/web/e2e/storefront.spec.js`; `playwright.config.mjs`; `.github/workflows/quality.yml`

Reasoning about the why

A rationale connects a choice to a constraint

A useful explanation identifies the problem, the chosen approach, the benefit under the project's constraints, the cost, and a condition that would make another approach preferable. Naming a technology without that reasoning does not explain the architecture.

Choices you should be able to discuss

SQLite: simple local setup and a real relational database suit a small assessment. The tradeoff is write concurrency and operating a shared database across multiple instances. PostgreSQL becomes more relevant if those requirements change.

Separate Express API: gives a clear HTTP boundary and reusable backend independent of the UI. It also means maintaining two processes, configuration sets, and cross-service requests. Using Next.js server routes instead would be a different boundary choice, not inherently incorrect.

Sequelize: organizes models and associations and reduces repetitive query plumbing. It introduces abstraction and generated-query behavior to understand. Repositories help localize that detail but do not make database migration effortless.

Database images: avoid needing a working external storage account for this assessment. They increase database and backup size and put asset requests through the API. The source images remain necessary for the current startup seed.

Context and feature components: fit a relatively small shared-state surface. Larger applications may need more precise subscriptions or a dedicated query cache. Splitting files should follow actual responsibilities.

State the limits plainly

The current order flow uses pay-on-delivery, not a payment gateway. Search/filter/sort on the backend currently load products and process them in application memory. Product-level stock is not separate per variant. Logout does not revoke already issued tokens. These limits are useful discussion points, not reasons to invent features.

Explain it back

Pick two choices and describe a scenario where you would replace them. If you cannot explain a drawback, revisit the code and the operational assumptions before calling the choice optimal.

Open in your repository: `apps/api/src/services/product.service.ts`; `repositories/product.repository.ts`; `services/order.service.ts`; `apps/web/src/components/providers`

Practice independently and document honestly

A closed-book explanation exercise

Close this guide. In your own notes, draw the path of a product read and an order submission. Label where input is validated, identity is verified, prices are read, and data is committed. Reopen the code to check your drawing. Repeat until you can explain the path without reading a prepared paragraph.

Questions for your three domains

Database: What relationships did you need? Why cents? Why migrations and a seed? How are images stored? What does a transaction protect?

Backend: What belongs in a controller versus a service? How is ownership enforced? How does checkout handle insufficient stock? What errors reach the client?

Frontend: Which parts run on the server or client? Where does shared state live? Why separate API adapters? How are empty, loading, and failed states handled?

Your AI-methodology account must come from your record

Reconstruct when AI entered the workflow, what you asked it to do, which suggestions or code you accepted, what you changed, and what you personally checked. Distinguish generated code, explanations, troubleshooting, configuration changes, documentation, and tests. Report AI-run tests separately from checks you personally performed.

In the visible conversation, AI helped implement the image-storage change, attempted R2 setup, centralized Prettier configuration, ran checks, and generated this study guide. That is only evidence from this conversation, not a complete account of your development. Review your full history yourself. Do not claim code, tests, or earlier documentation were manual if they were AI-generated.

Writing Markdown after you understand

Markdown is plain text with simple formatting: # starts a heading, - starts a list item, backticks surround inline code, and three backticks fence a code block. Use your own short paragraphs and concrete examples. You do not need polished language to explain a decision clearly. Do not ask AI to write, rewrite, or polish the deliverables that the assessment requires to be unaided.

A final self-check

Can you explain every technical term you use? Can you point to code supporting each implementation claim? Have you separated what exists from what you would improve? Does your stated development history match the evidence? If an explanation sounds impressive but you cannot demonstrate it, replace it with something you actually understand.

Reference basis: the repository files named throughout this guide and the installed Next.js documentation at `apps/web/node_modules/next/dist/docs/01-app/01-getting-started/02-project-structure.md`. No assessment submission text was authored as part of this guide.